

A Comparative Study of Machine Learning Models for Short-Term Solar Power Generation Forecasting Using Meteorological Data

1. Introduction

Background

Solar photovoltaic (PV) power is one of the fastest growing sources of electricity worldwide, but it is also one of the hardest to plan around. Unlike a coal or gas plant, a PV array only produces power when the sun is available, and its output changes constantly with cloud cover, ambient temperature and the angle of the sun through the day. For a grid operator this creates a real problem: too little forecasting accuracy and reserve generation has to be kept on standby “just in case,” which is expensive and, in the worst case, still not enough to prevent instability when solar output drops suddenly. Being able to predict solar output a few minutes to a day ahead, using nothing more than weather sensor readings, is therefore a genuinely useful capability, not just an academic exercise.

For a long time this forecasting problem was handled with physical models (Numerical Weather Prediction) or with classical statistics such as ARIMA. Both struggle here because the relationship between weather variables and power output is not a straight line — a small increase in irradiance near sunrise does not translate into the same power gain as a similar increase at midday. Machine learning models are better suited to this kind of non-linear relationship, and algorithms such as Random Forest, XGBoost and LSTM networks have all been used with reported success in recent papers. The problem, as the literature review below shows, is that these papers rarely agree with each other on which algorithm actually wins, mostly because they are run on different sites, different features and different train/test splits.

Aims and Objectives

The aim of this project is to compare Linear Regression, Random Forest, XGBoost and an LSTM network on the same solar dataset, under the same preprocessing and evaluation pipeline, to see which one actually forecasts short-term power output most accurately. To get there, the project set out to: review the recent literature and pin down where it agrees and disagrees; choose one public, well-documented dataset instead of trying to cover several; build a preprocessing pipeline that can be reproduced by someone else running the same notebook; implement and tune the four models listed above; evaluate them with MAE, RMSE and R^2 , including a breakdown by weather condition rather than one aggregate number; and finally look at which weather variables the models found most useful and what that means in practice.

Research Questions

Four questions guided the work: **RQ1** — which weather variables matter most for predicting short-term solar power; **RQ2** — do the tree-based ensembles (Random Forest, XGBoost) beat the LSTM, or the other way round; **RQ3** — does accuracy hold up equally well in clear-sky

conditions and in cloudy, more variable conditions; and **RQ4** — is the extra complexity of the tree ensembles and the LSTM actually worth it compared with a plain linear baseline.

Structure of the Report

Section 2 works through ten peer-reviewed papers on solar forecasting and pulls out the themes and the gap they leave open. Section 3 explains how the pipeline was built — the data, the tools, and how the models were judged. Section 4 summarises what was actually implemented in the notebook. Section 5 reports the results obtained by running that notebook on the real Kaggle dataset. Section 6 closes the report with what was achieved, the limitations that came up along the way, direct answers to the four research questions, and some suggestions for extending the work further.

2. Literature Review

Solar forecasting research is in an odd position: almost everyone agrees on which weather variables matter, yet almost no two papers agree on which algorithm is best. This section works through ten peer-reviewed studies by theme rather than one by one, to make that pattern — and the gap it leaves — clear.

Irradiance and temperature keep coming out on top. Across very different sites and climates — the Desert Knowledge Australia Solar Centre in Alice Springs (Aouidad and Bouhelal, 2024), a rooftop installation studied by Ledmaoui et al. (2023), and the site-level monitoring used by Zazoum (2022) — global horizontal irradiance shows up as the strongest predictor every time, usually close to a straight-line relationship with power output. Ambient and module temperature come in as useful secondary predictors, while wind speed, humidity and rainfall correlate more weakly and less consistently, though Aouidad and Bouhelal (2024) note they still matter somewhat through effects like panel cooling. This pattern is exactly why irradiation, ambient temperature and module temperature were chosen as the core feature set here, on top of the engineered time and lag features described in Section 3.

But nobody agrees on the best algorithm. Essam et al. (2022) found a plain Artificial Neural Network beating Random Forest, Decision Tree, XGBoost and LSTM. Balal et al. (2023), working with ten years of five-minute data from Lubbock, Texas, found the opposite pattern — Random Forest and LSTM came out ahead of Decision Tree, ANN, CNN and Gradient Boosting. Aouidad and Bouhelal (2024) themselves found Random Forest best when treated as a regression problem, but LSTM better once the same problem was turned into a four-class classification task. Zazoum (2022), on a different dataset again, reported Gaussian Process Regression as the strongest performer. Rahimi et al. (2023), reviewing a wide set of these studies, conclude that ensemble methods generally beat single learners, but that which specific model wins depends heavily on the site, the features chosen and how the evaluation is set up. That is really the gap this project is aimed at: rather than adding one more single-dataset ranking to a pile that already disagrees with itself, it runs four model families through one dataset, one pipeline and one set of metrics, so any difference found can reasonably be put down to the models rather than to inconsistent methodology.

Combining models tends to help. Several studies found that bagging, boosting, stacking or voting together outperformed any single learner on its own. Chakraborty et al. (2023), looking at a 10 kWp plant in Eastern India, found voting and stacking ensembles ahead of individual bagging and boosting models. Khan, Walker and Zeiler (2022) reported that stacking ANN, LSTM and XGBoost together improved R^2 by roughly 10–12% compared with any one of them alone, across several case studies. Persson et al. (2017) showed gradient boosted regression trees working well across multiple sites at once, and that paper is still cited as a foundation for a lot of the tree-based forecasting work that followed. This project only goes as far as including both a bagging-based ensemble (Random Forest) and a boosting-based one (XGBoost) rather than building a full stacked ensemble, but the pattern is consistent with what these studies found.

Good overall accuracy can still hide bad performance where it counts. Perhaps the most useful methodological point in the whole review comes from Aouidad and Bouhelal (2024): models with very high aggregate R^2 still failed to predict peak power output reliably, which is arguably the period that matters most for a grid operator trying to balance supply. An RMSE that looks excellent overall can be masking a model that is actually quite poor exactly when it needs to be good. That is the reasoning behind checking RMSE separately for clear-sky and cloudy/variable periods in this project (RQ3), rather than reporting one number and stopping there.

And the datasets themselves are all over the place. Resolution ranges from five minutes to hourly, duration from a few weeks to ten years, and geography from Australia to Texas to Eastern India to Denmark, with every study making its own choices about missing values, outliers and scaling. Rahimi et al. (2023) suggest this heterogeneity is itself a big reason the algorithm rankings never line up across studies. That is the argument for documenting every preprocessing decision in detail here (see Section 3), so whatever result comes out of this specific dataset can be read with a clear sense of the conditions that produced it — closing, at least for one dataset, the gap between what the literature agrees on (irradiance and temperature matter) and what it cannot agree on (which model is actually best).

3. Methodology

System Design

The pipeline follows a fairly standard supervised-learning structure. First, the 15-minute inverter-level generation data and the plant-level weather sensor data are merged on their timestamps, with generation summed across all inverters to give one total power reading per timestamp. Next comes exploratory analysis of missingness, distributions and correlations, followed by preprocessing: forward-filling any sensor gaps (using only past readings, not future ones, to avoid leaking information backward in time), clipping the handful of small negative power readings caused by sensor noise, and engineering calendar features (hour, day of week, month, plus a cyclical sine/cosine encoding of hour), lag features of power and irradiation, and rolling irradiation statistics. The data is then split chronologically — the most recent 20% of the timeline held out for testing — rather than shuffled randomly, since a random split would let the model train on timestamps that come after some of the test timestamps it is being judged on.

Tools and Technologies Used / Models Used

The notebook is written in Python, using pandas and NumPy for data handling, scikit-learn for Linear Regression, Random Forest and the TimeSeriesSplit/RandomizedSearchCV tuning, XGBoost for the boosting model, and TensorFlow/Keras for the LSTM. Matplotlib and seaborn handle the plots. The dataset is the public *Solar Power Generation Data* set on Kaggle (anikannal/solar-power-generation-data), specifically Plant 1: 15-minute inverter generation records and matching weather-sensor readings (irradiation, ambient and module temperature) covering 34 days.

Evaluation

All four models are scored on the same held-out test period using MAE, RMSE and R^2 . RMSE is then recalculated separately for clear-sky (high-irradiation) and cloudy/variable (low-irradiation) halves of the test set to check whether the aggregate number is hiding weaker performance somewhere specific, and Random Forest feature importances are pulled out to answer RQ1. Random Forest and XGBoost are also tuned with RandomizedSearchCV over a small hyperparameter grid, cross-validated with TimeSeriesSplit so that tuning itself never uses a future fold to score a past one.

4. Implementation

The pipeline was built as a single Jupyter notebook, split into clearly separated cells covering configuration, data loading, exploratory analysis, preprocessing and feature engineering, the chronological train/test split, feature scaling, model training and tuning, the LSTM, evaluation and visualisation, feature importance, and the regime-based error analysis, with a short auto-generated summary at the end. Each stage sits in its own function (`load_plant_data()`, `preprocess()`, `evaluate()`, and so on) so the notebook can be re-run on Plant 2 or adapted to a similarly structured dataset without much rewriting.

Unlike an earlier draft of this pipeline, the data-loading step no longer falls back to a synthetic dataset if the real Kaggle files are missing — it raises a clear error instead, so there is no risk of accidentally reporting results from fake data. A synthetic generator is still kept in an appendix cell purely so the rest of the code can be sanity-checked before the real files are downloaded, but it plays no part in the results below. XGBoost and TensorFlow are imported defensively with a fallback warning, so the notebook still runs (minus that one model) in an environment where either library is missing.

5. Results

The four models were run on Plant 1 of the real dataset: 3,157 merged 15-minute readings from 15 May to 17 June 2020, with no missing values in the merged file. After feature engineering and dropping the initial rows that lack full lag history, 3,061 rows remained, split chronologically into 2,448 training rows (16 May, 02:45 to 11 June, 14:00) and 613 test rows (11 June, 14:15 to 17 June, 23:45). Table 1 shows the test-set performance of all four models.

Model	MAE (kW)	RMSE (kW)	R ²
XGBoost	248.58	472.93	0.9965
Random Forest	225.55	474.64	0.9964
Linear Regression (baseline)	280.66	500.20	0.9960
LSTM	1313.05	2587.16	0.8939

XGBoost came out slightly ahead on RMSE, with Random Forest close behind and actually lower on MAE. Both beat the Linear Regression baseline, but only by a modest margin — about 5.4% lower RMSE than the baseline’s 500.20 kW. The LSTM, despite training for the full 100 epochs, was well behind all three other models, with an R² of 0.894 against 0.996 or higher for the rest — not what H2 in the research plan predicted.

Random Forest feature importance gave a clear answer to RQ1: IRRADIATION accounted for about 99.7% of total importance, dwarfing every other feature. The next four — a one-hour rolling mean of irradiation, the previous day’s lag of AC power, module temperature, and the rolling standard deviation of irradiation — each contributed under 0.05%. This lines up with the literature review’s first theme almost exactly: irradiance dominates, and the engineered lag/rolling features barely register once raw irradiation is available.

The regime breakdown for RQ3 told a more interesting story than the aggregate table alone. Every model’s RMSE was far higher during clear-sky (high-irradiation, high-output) periods than during cloudy/variable ones — for example, XGBoost’s RMSE was 667.35 kW under clear skies against just 35.40 kW when cloudy, and the same pattern held for the other three models. In other words, the very high R² values in Table 1 are mostly earned during the easier, low-output cloudy periods, and all four models actually struggle most when output is at its highest — exactly the concern Aouidad and Bouhelal (2024) raised about aggregate metrics hiding weak peak-period performance.

6. Conclusion

Achievements

The project delivered what it set out to: a literature review that identifies a genuine, specific gap (model rankings disagree across studies mainly because of inconsistent datasets and evaluation setups); a justified choice of one clean, public dataset instead of spreading effort across several; a documented, reproducible preprocessing pipeline with no data leakage between train and test; and four working models — a linear baseline, two tuned tree-based ensembles, and an LSTM — evaluated under identical conditions, with a regime-specific error breakdown on top of the usual aggregate metrics.

Limitations

A few limitations are worth being upfront about. The dataset covers only 34 days at a single plant, which is enough for a project of this size but does not say much about how these models would behave in a different climate or over a full year with real seasonal variation. The LSTM

used a fairly short two-hour input window and a simple architecture; a longer window or a more carefully tuned network might close some of the large gap seen in Section 5, so the current result should be read as “this particular LSTM setup underperformed,” not as a general statement about sequence models. The comparison is also limited to point forecasts — no uncertainty intervals — which matters less for a mini project but would matter for real grid planning. Finally, IRRADIATION and the temperature readings are recorded at the same timestamp as the power output being predicted, so this is closer to nowcasting (estimating current output from current weather) than a true day-ahead forecast that would need forecast, not observed, weather as input.

Answers to the Research Questions

RQ1: Irradiation is overwhelmingly the strongest predictor, matching almost every paper reviewed in Section 2. **RQ2:** the tree-based ensembles clearly beat the LSTM here — XGBoost and Random Forest both had RMSE under 475 kW against 2,587 kW for the LSTM — so H2’s prediction that the LSTM would have an edge on temporal dependencies did not hold with this window length and architecture. **RQ3:** no, performance is not even across conditions — every model’s error was much larger during high-output, clear-sky periods than during cloudy ones, so the strong aggregate R^2 values partly conceal weaker performance exactly when accurate forecasts matter most. **RQ4:** the added complexity was only partly worth it — Random Forest and XGBoost improved on the simple linear baseline by around 5%, which is a real but modest gain for the extra tuning effort involved, while the LSTM’s much higher training cost bought considerably worse accuracy, not better, in this run.

Future Work

A natural next step would be running the same pipeline on Plant 2 to see whether the model ranking holds up on a second site. Bringing in forecast weather data instead of observed readings would turn this from a nowcasting exercise into an actual forward-forecasting tool. A properly tuned LSTM (longer window, wider search over architecture) would also be worth trying before drawing any firm conclusion about deep learning’s place in this comparison. Given how strongly the literature review favours combining models, a stacked ensemble of Random Forest, XGBoost and the LSTM is another obvious direction. Finally, moving from point forecasts to a probabilistic approach — quantile regression, for instance — would make the output more directly useful for grid operational planning.

Harvard Reference List

- Aouidad, H.I. and Bouhelal, A. (2024) ‘Machine learning-based short-term solar power forecasting: a comparison between regression and classification approaches using extensive Australian dataset’, *Sustainable Energy Research*, 11, article 28. <https://doi.org/10.1186/s40807-024-00115-1>
- Balal, A., Jafarabadi, Y.P., Demir, A., Igene, M., Giesselmann, M. and Bayne, S. (2023) ‘Forecasting solar power generation utilizing machine learning models in Lubbock’, *Emerging Science Journal*, 7(4), pp. 1052–1062. <https://doi.org/10.28991/ESJ-2023-07-04-02>

Chakraborty, D., Mondal, J., Barua, H.B. and Bhattacharjee, A. (2023) 'Computational solar energy—Ensemble learning methods for prediction of solar power generation based on meteorological parameters in Eastern India', *Renewable Energy Focus*, 44, pp. 277–294. <https://doi.org/10.1016/j.ref.2023.01.006>

Essam, Y., Ahmed, A.N., Ramli, R., Chau, K.W., Idris Ibrahim, M.S., Sherif, M., Sefelnasr, A. and El-Shafie, A. (2022) 'Investigating photovoltaic solar power output forecasting using machine learning algorithms', *Engineering Applications of Computational Fluid Mechanics*, 16(1), pp. 2002–2034. <https://doi.org/10.1080/19942060.2022.2126528>

Khan, W., Walker, S. and Zeiler, W. (2022) 'Improved solar photovoltaic energy generation forecast using deep learning-based ensemble stacking approach', *Energy*, 240, 122812. <https://doi.org/10.1016/j.energy.2021.122812>

Ledmaoui, Y., El Maghraoui, A., El Aroussi, M., Saadane, R., Chebak, A. and Chehri, A. (2023) 'Forecasting solar energy production: A comparative study of machine learning algorithms', *Energy Reports*, 10, pp. 1004–1012. <https://doi.org/10.1016/j.egyr.2023.07.042>

Persson, C., Bacher, P., Shiga, T. and Madsen, H. (2017) 'Multi-site solar power forecasting using gradient boosted regression trees', *Solar Energy*, 150, pp. 423–436. <https://doi.org/10.1016/j.solener.2017.04.066>

Rahimi, N., Park, S., Choi, W., Oh, B., Kim, S., Cho, Y.H., Ahn, S., Chong, C., Kim, D., Jin, C. and Lee, D. (2023) 'A comprehensive review on ensemble solar power forecasting algorithms', *Journal of Electrical Engineering and Technology*, 18(2), pp. 719–733. <https://doi.org/10.1007/s42835-23-01378-2>

Zazoum, B. (2022) 'Solar photovoltaic power prediction using different machine learning methods', *Energy Reports*, 8, pp. 19–25. <https://doi.org/10.1016/j.egyr.2021.11.183>

Dataset and Notebook Links

Dataset (Kaggle): <https://www.kaggle.com/datasets/anikannal/solar-power-generation-data>

Notebook (Google Colab):

[https://colab.research.google.com/drive/1AO9PhvE1kiFY8knKtg9Pb5gQ4zMx5TIP?](https://colab.research.google.com/drive/1AO9PhvE1kiFY8knKtg9Pb5gQ4zMx5TIP?usp=sharing)
[usp=sharing](https://colab.research.google.com/drive/1AO9PhvE1kiFY8knKtg9Pb5gQ4zMx5TIP?usp=sharing)